

Recenzja Specyfikacji Wymagań Aplikacji "FlatShare"

Recenzent – zespół "Kaloriusz" w składzie:
Mikołaj Złotek, Maciej Kondratowicz, Jan Olech, Maksim Hamanchuk

28 stycznia 2026

1 Wstęp i ogólny opis specyfikacji

Recenzja dotyczy finalnej wersji dokumentacji projektowej systemu "FlatShare", przygotowanej przez zespół w składzie: Jakub Rak, Bartosz Radomski, Sofia Kuzmenko oraz Piotr Wójcik.

1.1 Cel i zakres aplikacji

Projektowana aplikacja ma na celu bezpośrednie połączenie właścicieli mieszkań z potencjalnymi lokatorami. Główną wartością biznesową systemu jest zastosowanie zaawansowanych mechanizmów dopasowania, które analizują specyficzne cechy oraz indywidualne preferencje obu stron. Zamiast prezentowania pełnej, niefiltrowanej listy ofert, system dostarcza użytkownikowi zestawienie propozycji najlepiej odpowiadających jego potrzebom i osobliwościom. Poza tym integralną częścią aplikacji jest moduł łatwej i bezpośredniej komunikacji. Rozwiązanie to nie tylko przyspiesza proces najmu, ale również znacząco zmniejsza ryzyko oszustw, pozwalając na bezpieczną wymianę informacji i wszelką weryfikację wewnątrz platformy.

Kluczowe komponenty systemu obejmują:

- **Moduł ogłoszeń i wyszukiwania** – umożliwiają właścicielom publikację ofert, a najemcom ich przeglądanie z wykorzystaniem zaawansowanych filtrów.
- **Silnik dopasowania** – algorytm obliczający stopień zgodności profilu najemcy z ofertą właściciela.
- **System rezerwacji i płatności** – obsługujący proces od złożenia zapytania, przez płatność online, aż po finalne potwierdzenie wynajmu.
- **Komunikator/Chat** – narzędzie do bezpośredniej, bezpiecznej komunikacji między stronami wewnątrz aplikacji.

1.2 Struktura dokumentacji

Recenzowany dokument charakteryzuje się bardzo dobrą strukturą hybrydową. Treść została logicznie podzielona na dwa główne bloki:

1. **Część analityczna i projektowa** – jest to fundament dla zrozumienia architektury systemu. Obejmuje ona takie podsekcje jak:
 - wprowadzenie oraz definicję celu biznesowego i granic systemu;
 - wymagania funkcjonalne opisane za pomocą Historyjek Użytkownika (User Stories) wraz z precyzyjnymi Kryteriami Akceptacji;
 - wysokopoziomowe modelowanie systemu, zawierające diagramy przypadków użycia, klas, stanów oraz aktywności.
2. **Szczegółowa specyfikacja procesów** – zorganizowana według kluczowych funkcjonalności systemu (takich jak: Logowanie, Wystawianie ogłoszenia, Zarządzanie preferencjami, Wynajem, Komunikacja, Moderacja). W tej części autorzy odeszli od ogólnego opisu na rzecz pełnej agregacji technicznej. Każda sekcja funkcjonalna zawiera kompletny zestaw informacji:
 - dedykowane diagramy i opisy przepływu dla danego procesu;
 - specyfikację endpointów API (metody HTTP, ścieżki) oraz wymogi bezpieczeństwa (JWT);
 - konkretne przykłady struktur danych (JSON) oraz obsługi błędów.

Taki układ dokumentu, w którym szczegóły techniczne są bezpośrednio przypisane do omawianych procesów biznesowych, jest naszym zdaniem bardzo czytelny i ułatwia weryfikację spójności rozwiązania.

2 Szczegółowe uwagi i elementy wymagające poprawy

Mimo wysokiej jakości dostarczonej dokumentacji, identyfikujemy obszary wymagające doprecyzowania lub korekty logicznej. Poniżej przedstawiamy szczegółową listę elementów, które naszym zdaniem wymagają poprawy przed przystąpieniem do implementacji.

2.1 Brak procedury odwołania rezerwacji w przypadku blokady właściciela

W sekcji 14.3 opisano mechanizm blokowania użytkownika (Właściciela) przez Administratora. Dokumentacja wskazuje, że blokada skutkuje ukryciem ogłoszeń, jednak brakuje informacji o obsłudze ****aktywnych i opłaconych rezerwacji**** (status **CONFIRMED**) u zablokowanego użytkownika.

Zagrożenie biznesowe: Pozostawienie aktywnych rezerwacji u właściciela uznanego za oszusta lub naruszającego regulamin zwiększa ryzyko tego, że lokator zostanie oszukany lub niezadowolony warunkami wynajętego mieszkania.

Rekomendacja: Należy uzupełnić Diagram Stanów Rezerwacji o przejście awaryjne (wymuszone przez system/admina), które anuluje przyszłe rezerwacje i automatycznie inicjuje proces zwrotu środków (np. Refund). Potencjalnie system może tylko poinformować lokatorów, które mają opłacone przyszłe rezerwacje, że szansa oszustwa jest większa od zwykłej i że można podjąć odpowiednią decyzję (anulowanie i próba zwrotu pieniędzy).

2.2 Brak obsługi czasowej niedostępności

Analiza modelu danych (ERD) oraz sekcji zarządzania ogłoszeniem wskazuje na brak mechanizmu pozwalającego właścicielowi na czasowe wyłączenie mieszkania z wynajmu bez tworzenia rezerwacji. (Oczywiście razem z tym trzeba będzie dodać mechanizm ponownego włączania mieszkania do wynajmu z powrotem).

Przykład: Do właściciela mieszkania przyjeżdżają jego przyjaciele z innego miasta na miesiąc. Chce on na ten okres wyłączyć mieszkanie z wynajmu. Obecny model wymusza właściciela na albo usunięcie ogłoszenia, albo dokonanie "fikcyjnej" rezerwacji samego siebie, co pogarsza jakość danych analitycznych.

Rekomendacja: Sugerujemy dodanie encji `Unavailability` powiązanej z `Listing`, definiującej zakres dat wyłączonych z dostępności.

2.3 Niespójności w logice rezerwacji: Ryzyko wyścigu oraz Zakleszczenia

Analiza procesu rezerwacji (Sekcja 4.2) oraz diagramu stanów (Rysunek 10) ujawniła niespójność w zarządzaniu dostępnością zasobów. Problem składa się z dwóch powiązanych aspektów:

- **Problem 1: Wyścig.** Zgodnie ze scenariuszem głównym, system blokuje termin w kalendarzu dopiero **po** potwierdzeniu płatności (status `CONFIRMED`). Stwarza to ryzyko *overbookingu* – dwóch użytkowników może równolegle wejść w proces płatności za ten sam termin i obaj zostaną obciążeni finansowo, mimo że dostępne jest tylko jedno miejsce.
- **Rekomendacja naprawy:** Blokada terminu (rezerwacja tymczasowa) musi nastąpić wcześniej – już w momencie inicjacji płatności (wejście w stan `PENDING_PAYMENT`).
- **Problem 2: Ryzyko zakleszczenia jako skutek naprawy.** Przesunięcie blokady na wcześniejszy etap wprowadza nowe zagrożenie. Jeśli użytkownik zablokuje termin (wchodząc w `PENDING_PAYMENT`), a następnie porzuci proces bez płacenia, termin pozostanie zablokowany w nieskończoność, ponieważ diagram stanów `Booking` nie przewiduje mechanizmu `timeout` dla tego stanu.

Wniosek: Konieczne jest wprowadzenie mechanizmu **rezerwacji tymczasowej z limitem czasu** (np. 15 minut). Jeśli płatność nie nastąpi w tym oknie, system musi automatycznie zwolnić blokadę terminu i zmienić status rezerwacji na `EXPIRED`.

2.4 Uwagi redakcyjne i błędy w kodowaniu znaków

W dokumentacji zidentyfikowano szereg błędów edycyjnych, które obniżają czytelność specyfikacji technicznej:

- **Kodowanie znaków w przykładach JSON:** W listingach kodu (np. Listing 9, Listing 25) polskie znaki są usuwane lub błędnie kodowane (np. `"title": "Pokj przy metrze"` zamiast `"Pokój"`).

3 Pozytywne aspekty specyfikacji

Mimo wskazanych powyżej uwag krytycznych, pragniemy wyraźnie podkreślić, że recenzowana dokumentacja reprezentuje naprawdę wysoki poziom inżynierski. Za najmocniejsze strony projektu uważamy:

- **Modułową agregację wiedzy technicznej.** Spodobało się nam odejście od sztucznej separacji diagramów i kodu na rzecz opisu procesowego. Umieszczenie modelu danych, definicji endpointów API, przykładów JSON oraz diagramów sekwencji w ramach jednego rozdziału funkcjonalnego (np. w sekcji *Proces rezerwacji*) znacząco ułatwia analizę i późniejszą implementację systemu.
- **Szczegółowe modelowanie diagramów.** Różnego rodzaju diagramy są wyczerpujące. Autorzy uwzględnili nie tylko tzw. "szczęśliwe ścieżki", ale również scenariusze negatywne, co świadczy o skrupulatnej analizie przypadków brzegowych.
- **Elastyczny model ról użytkowników.** W warstwie projektowej (Diagram Klas) zastosowano kompozycję zamiast dziedziczenia do obsługi ról (Właściciel/Lokator). Takie podejście pozwala na łatwe rozszerzanie systemu (jeden użytkownik może być jednocześnie właścicielem i najemcą) i jest zgodne z najlepszymi wzorcami programowania obiektowego.
- **Dedykowany moduł bezpieczeństwa społeczności.** Na duży plus zasługuje zaprojektowanie pełnego obiegu zgłoszeń naruszeń (*ViolationReport*). Zespół nie pominął aspektu moderacji treści, definiując endpointy i procesy pozwalające na walkę z nieuczciwymi użytkownikami, co w aplikacji typu *social-renting* jest fundamentem zaufania.

4 Podsumowanie

Dokumentacja projektu „FlatShare” stanowi naszym zdaniem solidną podstawę do realizacji systemu informatycznego. Struktura specyfikacji, szczegółowość API oraz dojrzałość diagramów UML świadczą o wspaniałej wiedzy technicznej zespołu. Rekomendujemy przyjęcie specyfikacji pod warunkiem naniesienia poprawek w kilku miejscach, gdzie według naszej opinii może potencjalnie zostać naruszona logika biznesowa.